

11 Inheritance and Polymorphism



Motivations

Suppose you will define classes to model circles, rectangles, and triangles. These classes **have many common features**.

What is the best way to design these classes so **to avoid redundancy**?

The answer is to use inheritance.



Objectives

- To define a subclass from a superclass through inheritance (§11.2).
- To invoke the superclass's constructors and methods using the **super** keyword (§11.3).
- To override instance methods in the subclass (§11.4).
- To distinguish differences between overriding and overloading (§11.5).
- To explore the **toString()** method in the **Object** class (§11.6).
- To discover polymorphism and dynamic binding (§§11.7–11.8).
- To describe casting and explain why explicit downcasting is necessary (§11.9).
- To explore the **equals** method in the **Object** class (§11.10).
- To store, retrieve, and manipulate objects in an **ArrayList** (§11.11).
- To implement a **Stack** class using **ArrayList** (§11.12).
- To enable data and methods in a superclass accessible from subclasses using the **protected** visibility modifier (§11.13).
- To prevent class extending and method overriding using the **final** modifier (§11.14).



Superclasses and Subclasses

GeometricObject
-color: String -filled: boolean -dateCreated: java.util.Date
+GeometricObject() +GeometricObject(color: String, filled: boolean) +getColor(): String +setColor(color: String): void +isFilled(): boolean +setFilled(filled: boolean): void +getDateCreated(): java.util.Date +toString(): String

The color of the object (default: white).

Indicates whether the object is filled with a color (default: false).

The date when the object was created.

Creates a GeometricObject.

Creates a GeometricObject with the specified color and filled values.

Returns the color.

Sets a new color.

Returns the filled property.

Sets a new filled property.

Returns the dateCreated.

Returns a string representation of this object.

Circle
-radius: double
+Circle() +Circle(radius: double) +Circle(radius: double, color: String, filled: boolean) +getRadius(): double +setRadius(radius: double): void +getArea(): double +getPerimeter(): double +getDiameter(): double +printCircle(): void

Rectangle
-width: double -height: double
+Rectangle() +Rectangle(width: double, height: double) +Rectangle(width: double, height: double, color: String, filled: boolean) +getWidth(): double +setWidth(width: double): void +getHeight(): double +setHeight(height: double): void +getArea(): double +getPerimeter(): double

GeometricObject

CircleFromSimpleGeometricObject

RectangleFromSimpleGeometricObject

TestCircleRectangle

Run

Are superclass's Constructor Inherited?

No. They are not inherited.

They are invoked explicitly or implicitly.

Explicitly using the `super` keyword.

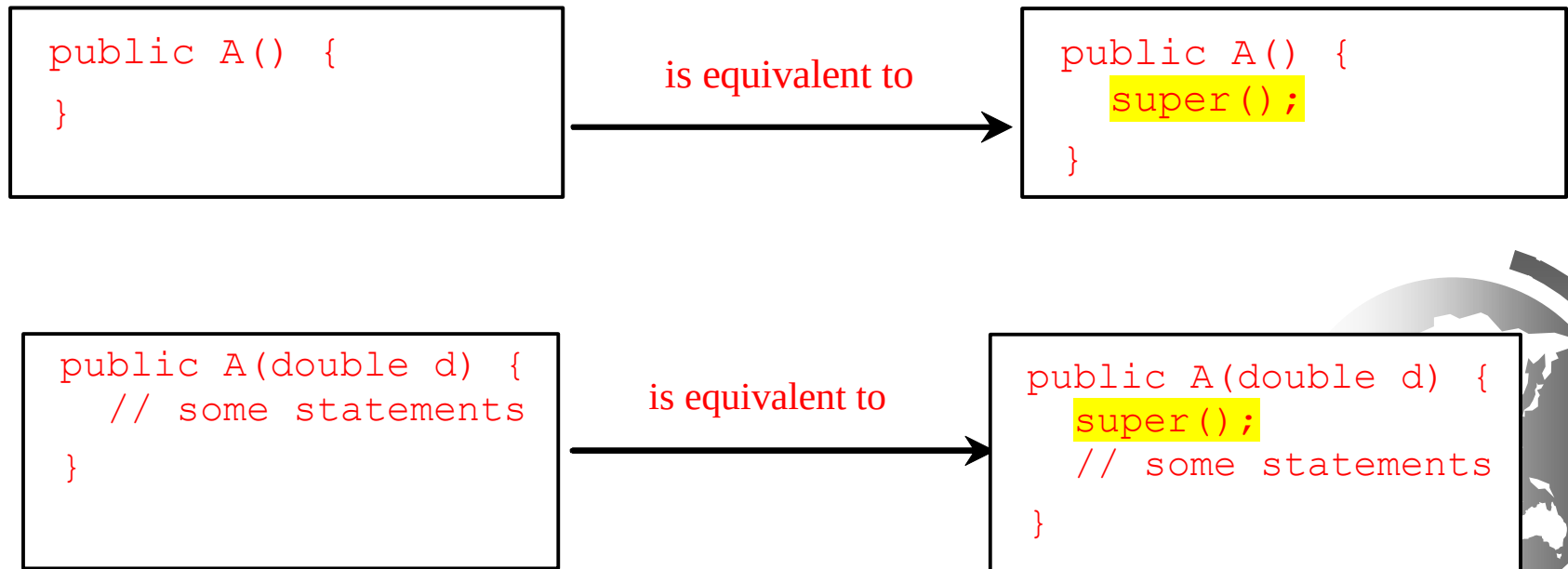
A constructor is used to construct an instance of a class. Unlike properties and methods, **a superclass's constructors are not inherited in the subclass**. They can only be invoked from the subclasses' constructors, using the keyword `super`.

If the keyword `super` is not explicitly used, the superclass's no-arg constructor is automatically invoked.



Superclass's Constructor Is Always Invoked

A constructor may **invoke** an overloaded constructor **or** its **superclass's constructor**. If none of them is invoked explicitly, the compiler puts super() as the **first statement** in the constructor. For example,



Using the Keyword `super`

The keyword `super` refers to the superclass of the class in which `super` appears. This keyword can be used in two ways:

- To call a superclass constructor

- To call a superclass method



CAUTION

You must use the keyword super to call the superclass constructor.

Invoking a superclass constructor's name in a subclass causes a **syntax error**. Java requires that the statement that uses the keyword super appear first in the constructor.



Constructor Chaining

Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain. This is known as *constructor chaining*.

```
public class Faculty extends Employee {
    public static void main(String[] args) {
        new Faculty();
    }

    public Faculty() {
        System.out.println("(4) Faculty's no-arg constructor is invoked");
    }
}

class Employee extends Person {
    public Employee() {
        this("(2) Invoke Employee's overloaded constructor");
        System.out.println("(3) Employee's no-arg constructor is invoked");
    }

    public Employee(String s) {
        System.out.println(s);
    }
}

class Person {
    public Person() {
        System.out.println("(1) Person's no-arg constructor is invoked");
    }
}
```



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

1. Start from the
main method



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

2. Invoke Faculty
constructor



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

3. Invoke Employee's no-arg constructor



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

4. Invoke Employee(String)
constructor



Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

5. Invoke Person() constructor

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

6. Execute println

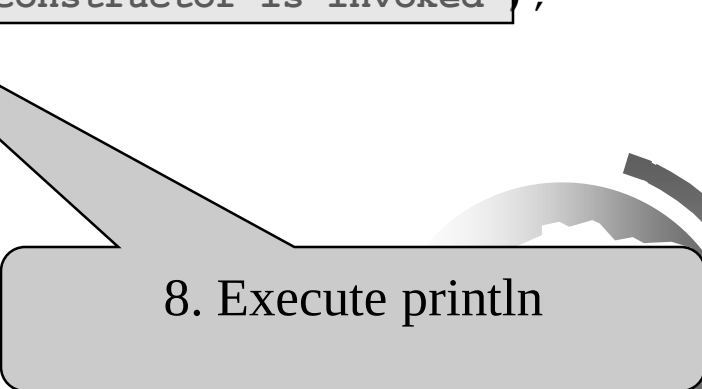
Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

7. Execute println

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



8. Execute println

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

9. Execute println



Example on the Impact of a Superclass without no-arg Constructor

Find out the errors in the program:

```
public class Apple extends Fruit {  
}  
  
class Fruit {  
    public Fruit(String name) {  
        System.out.println("Fruit's constructor is invoked");  
    }  
}
```



What will happen if you try to compile and execute B's `main()` method? ()

```
class A {  
    int i;  
    A(int i) {        this.i = i * 2; }  
}  
class B extends A {  
    public static void main(String[] args) {  
        B b = new B(2);  
    }  
    B(int x) {  
        System.out.println(x);  
    }  
}
```

- ☐ A. The instance variable i is set to 4
- ☐ B. The instance variable i is set to 2
- ☐ C. The instance variable i is set to 0
- ☒ D. This code will not compile



Defining a Subclass

A subclass inherits from a superclass. You can also:

- Add new properties

- Add new methods

- Override the methods of the superclass



Calling Superclass Methods

You could rewrite the printCircle() method in the Circle class as follows:

```
public void printCircle() {  
    System.out.println("The circle is created " +  
        super.getDateCreated() + " and the radius is " + radius);  
}
```



Overriding Methods in the Superclass

A subclass inherits methods from a superclass. Sometimes it is necessary for the subclass to **modify the implementation** of a method defined in the superclass. This is referred to as *method overriding*.

```
public class Circle extends GeometricObject {  
    // Other methods are omitted  
  
    /** Override the toString method defined in GeometricObject */  
    public String toString() {  
        return super.toString() + "\nradius is " + radius;  
    }  
}
```



NOTE

An instance method can be overridden **only if it is accessible**.

Thus **a private method cannot be overridden**, because it is not accessible outside its own class.

If a method defined in a subclass is **private** in its superclass, the two methods are completely unrelated.



NOTE

Like an instance method, a static method can be inherited.

However, a static method cannot be overridden.

If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden.



// Can you spot the bug?

```
public class Bigram {  
    private final char first;  
    private final char second;  
    public Bigram(char first, char second) {  
        this.first = first;  
        this.second = second;  
    }  
    public boolean equals(Bigram b) {  
        return b.first == first && b.second == second;  
    }  
    public int hashCode() {  
        return 31 * first + second;  
    }  
  
    public static void main(String[] args) {  
        Set<Bigram> s = new HashSet<Bigram>();  
        for (int i = 0; i < 10; i++)  
            for (char ch = 'a'; ch <= 'z'; ch++)  
                s.add(new Bigram(ch, ch));  
        System.out.println(s.size());  
    }  
}
```

Override?
Overload?

NOTE

- 坚持使用Override注解
- Java1.5中增加Override注解，可防止前面的非法错误

```
@override public boolean equals(Bigram b){  
    return b.first==first && b.second==second;  
}
```

编译错误

```
Bigram.java:10: method does not override or implement a method  
from a supertype
```

```
    @Override public boolean equals(Bigram b) {  
    ^
```

```
@Override public boolean equals(Object o) {  
    if (!(o instanceof Bigram))  
        return false;  
    Bigram b = (Bigram) o;  
    return b.first == first && b.second == second;  
}
```

Overriding vs. Overloading

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overrides the method in B  
    public void p(double i) {  
        System.out.println(i);  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overloads the method in B  
    public void p(int i) {  
        System.out.println(i);  
    }  
}
```

方法重写发生在通过继承而相关的不同类中；而方法重载可以发生在同一个类中，也可以发生在由于继承而相关的不同类中。

- overload(重载)方法的选择是静态的，而对于被覆盖的方法(overridden method)的选择则是动态的。

```
// Broken! - What does this program print?
public class CollectionClassifier {
    public static String classify(Set<?> s) {
        return "Set";
    }

    public static String classify(List<?> lst) {
        return "List";
    }

    public static String classify(Collection<?> c) {
        return "Unknown Collection";
    }

    public static void main(String[] args) {
        Collection<?>[] collections = {
            new HashSet<String>(),
            new ArrayList<BigInteger>(),
            new HashMap<String, String>().values()
        };

        for (Collection<?> c : collections)
            System.out.println(classify(c));
    }
}
```

打印“Unknown
Collection” 三次

对于参数的匹配
， overload， 静态
匹配



The Object Class and Its Methods

Every class in Java is descended from the `java.lang.Object` class. If no inheritance is specified when a class is defined, the superclass of the class is `Object`.

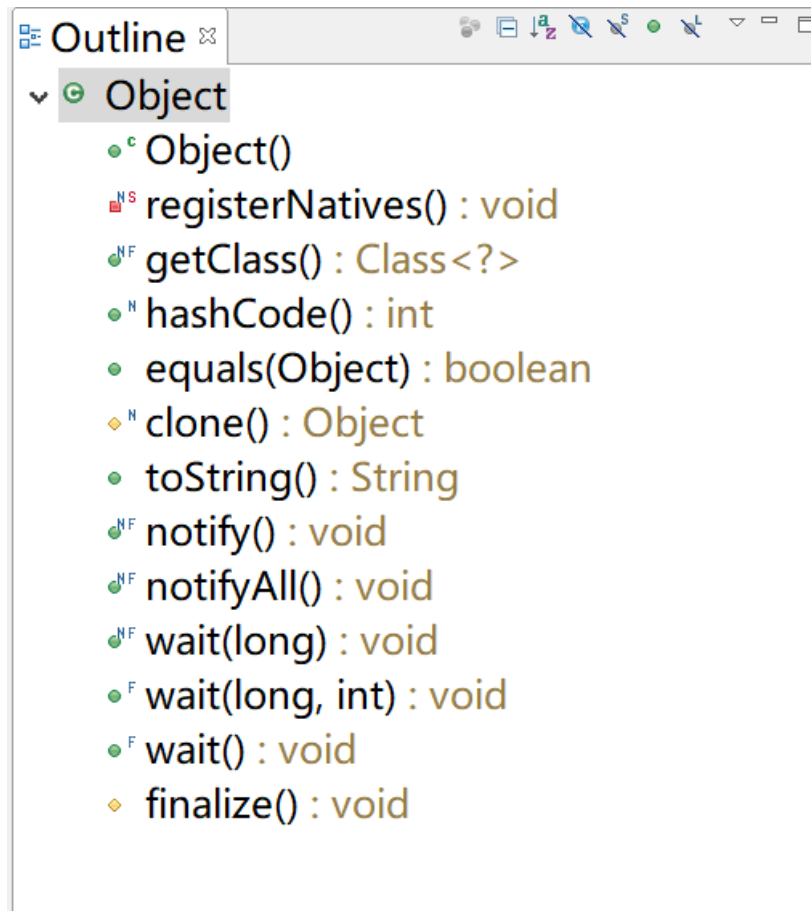
```
public class Circle {  
    ...  
}
```

Equivalent

```
public class Circle extends Object {  
    ...  
}
```



Methods in Object Class



The toString() method in Object

The toString() method returns a string representation of the object. **The default implementation returns a string consisting of a class name of which the object is an instance, the at sign (@), and a number representing this object.**

```
Loan loan = new Loan();
```

```
System.out.println(loan.toString());
```

The code displays something like `Loan@15037e5` . **This message is not very helpful or informative. Usually you should **override** the toString method so that it returns a digestible string representation of the object.**

```
public String toString() {  
    return getClass().getName() + "@" + Integer.toHexString(hashCode());  
}
```


Polymorphism

Polymorphism means that **a variable of a supertype can refer to a subtype object.**

A class defines a type. A type defined by a subclass is called a *subtype*, and a type defined by its superclass is called a *supertype*.

Therefore, you can say that **Circle** is a subtype of **GeometricObject** and **GeometricObject** is a supertype for **Circle**.



PolymorphismDemo

Run

Polymorphism means that **a variable of a supertype can refer to a subtype object.**

SimpleGeometricObject obj = new CircleFromSimpleGeometricObject(...)

SimpleGeometricObject obj = new RectangleFromSimpleGeometricObject(...)

```
1 public class PolymorphismDemo {
2     /** Main method */
3     public static void main(String[] args) {
4         // Display circle and rectangle properties
5         displayObject(new CircleFromSimpleGeometricObject
6             (1, "red", false));
7         displayObject(new RectangleFromSimpleGeometricObject
8             (1, 1, "black", true));
9     }
10
11     /** Display geometric object properties */
12     public static void displayObject(SimpleGeometricObject object) {
13         System.out.println("Created on " + object.getDateCreated() +
14             ". Color is " + object.getColor());
15     }
16 }
17
```

Problems @ Javadoc Declaration Console

```
<terminated> PolymorphismDemo [Java Application] C:\Program Files\Java\jdk1.8.0_141\jre
Created on Wed Oct 16 20:55:02 CST 2019. Color is red
Created on Wed Oct 16 20:55:02 CST 2019. Color is black
```

Polymorphism, Dynamic Binding and Generic Programming

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}  
  
class GraduateStudent extends Student {  
}  
  
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}  
  
class Person extends Object {  
    public String toString() {  
        return "Person";  
    }  
}
```

Method m takes a parameter of the Object type. You can invoke it with any object.

An object of a subtype can be used wherever its supertype value is required. This feature is known as *polymorphism*.

When the method m(Object x) is executed, the argument x's toString method is invoked. x may be an instance of GraduateStudent, Student, Person, or Object. Classes GraduateStudent, Student, Person, and Object have their own implementation of the toString method.

Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime. This capability is known as *dynamic binding*.

Animation

DynamicBindingDemo

Run

```

3 public class PolymorphismDemo {
4     /** Main method */
5     public static void main(String[] args) {
6         m(new GraduateStudent());
7         m(new Stud());
8         m(new Person());
9         m(new Object());
10
11     }
12
13     public static void m(Object x) {
14         System.out.println(x.toString());
15     }
16 }
17
18 class GraduateStudent extends Stud {
19 }
20
21 class Stud extends Person {
22     public String toString() {
23         return "Student";
24     }
25 }
26
27 class Person extends Object {
28     public String toString() {
29         return "Person";
30     }
31 }

```

<terminated> PolymorphismDemo (1) [Java Application]

Student

Student

Person

java.lang.Object@15db9742

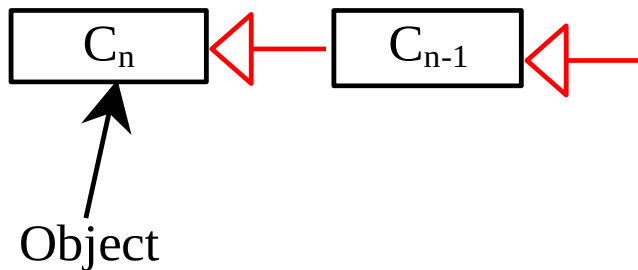


Dynamic Binding

Dynamic binding works as follows: Suppose an object **o** is an instance of classes $C_1, C_2, \dots, C_{n-1}$, and C_n , where C_1 is a subclass of C_2 , C_2 is a subclass of C_3 , ..., and C_{n-1} is a subclass of C_n . That is, C_n is the most general class, and C_1 is the most specific class. In Java, C_n is the Object class.

If **o** invokes a method **p**, the **JVM** searches the implementation for the method **p** in $C_1, C_2, \dots, C_{n-1}$ and C_n , in this order, until it is found. Once an implementation is found, the search stops and the first-found implementation is invoked.

$C_n \text{ } o = \text{new } C_1()$



Since o is an instance of C_1 , o is also an instance of $C_2, C_3, \dots, C_{n-1}$, and C_n

Method Matching vs. Binding

Matching a method signature and **binding** a method implementation are two issues.

The **compiler** finds a matching method according to parameter type, number of parameters, and order of the parameters at **compilation time**.

A method may be implemented in several subclasses.

The **Java Virtual Machine** dynamically binds the implementation of the method at **runtime**.



```

3 public class PolymorphismDemo {
4     /** Main method */
5     public static void main(String[] args) {
6         m(new GraduateStudent());
7         m(new Stud());
8         m(new Person());
9         m(new Object());
10
11     }
12
13     public static void m(Object x) {
14         System.out.println(x.toString());
15     }
16     public static void m(Stud s) {
17         System.out.println("In m(Stud s): " + s.toString());
18     }
19 }
20
21 class GraduateStudent extends Stud {
22 }
23
24 class Stud extends Person {
25     public String toString() {
26         return "Student";
27     }
28 }
29
30 class Person extends Object {
31     public String toString() {
32         return "Person";
33     }
34 }

```

<terminated> PolymorphismDemo (1) [Java Application] C:\Program Files\Java\jdk1.8.0_141\jre

In m(Stud s): Student

In m(Stud s): Student

Person

java.lang.Object@15db9742

Generic Programming

```
public class PolymorphismDemo {
    public static void main(String[] args) {
        m(new GraduateStudent());
        m(new Student());
        m(new Person());
        m(new Object());
    }

    public static void m(Object x) {
        System.out.println(x.toString());
    }
}

class GraduateStudent extends Student {
}

class Student extends Person {
    public String toString() {
        return "Student";
    }
}

class Person extends Object {
    public String toString() {
        return "Person";
    }
}
```

Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as **generic programming**.

If a method's parameter type is a superclass (e.g., Object), you may pass an object to this method of any of the parameter's subclasses (e.g., Student or String).

When an object (e.g., a Student object or a String object) is used in the method, the particular implementation of the method of the object that is invoked (e.g., toString) is determined dynamically.



Casting Objects

You have already used the casting operator to convert variables of one primitive type to another. *Casting can also be used to convert an object of one class type to another within an inheritance hierarchy.*

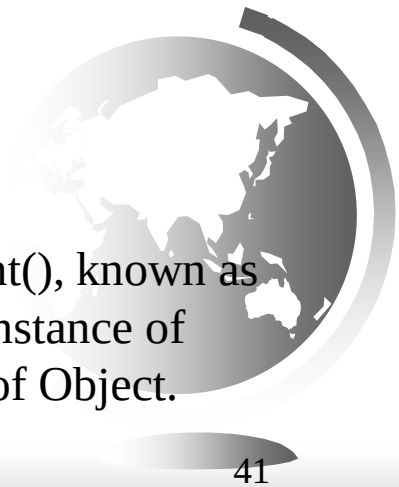
In the preceding section, the statement

```
m(new Student());
```

assigns the object new Student() to a parameter of the Object type. This statement is equivalent to:

```
Object o = new Student(); // Implicit casting  
m(o);
```

The statement Object o = new Student(), known as implicit casting, is legal because an instance of Student is automatically an instance of Object.



Why Casting Is Necessary?

Suppose you want to assign the object reference `o` to a variable of the `Student` type using the following statement:

`Student b = o;` **A compile error would occur.**

Why does the statement **`Object o = new Student()`** work and the statement **`Student b = o`** doesn't?

This is because a `Student` object is always an instance of `Object`, but an `Object` is not necessarily an instance of `Student`. Even though you can see that `o` is really a `Student` object, **the compiler is not so clever to know it.**

To tell the compiler that `o` is a `Student` object, use an **explicit casting**. The syntax is similar to the one used for casting among primitive data types:

`Student b = (Student)o; // Explicit casting`



Casting from Superclass to Subclass

Explicit casting must be used when casting an object from a superclass to a subclass. This type of casting **may not always succeed**.

```
Apple x = (Apple) fruit;
```

```
Orange x = (Orange) fruit;
```



The instanceof Operator

Use the `instanceof` operator to test whether an object is an instance of a class:

```
Object myObject = new Circle();  
... // Some lines of code  
/** Perform casting if myObject is an instance of  
    Circle */  
if (myObject instanceof Circle) {  
    System.out.println("The circle diameter is " +  
        ((Circle)myObject).getDiameter());  
    ...  
}
```



TIP

To help understand casting, you may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange.

An apple is a fruit, so you can always safely assign an instance of Apple to a variable for Fruit. However, a fruit is not necessarily an apple, so you have to use explicit casting to assign an instance of Fruit to a variable of Apple.



Java VS. C++

- `Manager boss = (Manager)staff; //java`
- 类似于C++中的 `Manager* boss = dynamic_cast<Manager*>(staff);`
- 之前一个重要的区别在于：当类型转换失败时，Java不会生成一个null对象，而是抛出一个异常；而在C++中，则会生成null对象，这样可以在一个操作中完成类型测试和类型转换

```
Manager *boss = dynamic_cast<Manager*>staff;
```

```
If(boss !=NULL)....
```

在Java中，则需要用instanceof和类型转换结合起来使用：

```
If(staff instanceof Manager)
{
    Manager boss = (Manager)staff;
}
```



Example: Demonstrating Polymorphism and Casting

This example creates two geometric objects: a circle, and a rectangle, invokes the `displayGeometricObject` method to display the objects. The `displayGeometricObject` displays the area and diameter if the object is a circle, and displays area if the object is a rectangle.



CastingDemo

Run



```

1 public class CastingDemo {
2     /** Main method */
3     public static void main(String[] args) {
4         // Create and initialize two objects
5         Object object1 = new CircleFromSimpleGeometricObject(1);
6         Object object2 = new RectangleFromSimpleGeometricObject(1, 1);
7
8         // Display circle and rectangle
9         displayObject(object1);
10        displayObject(object2);
11    }
12
13    /** A method for displaying an object */
14    public static void displayObject(Object object) {
15        if (object instanceof CircleFromSimpleGeometricObject) {
16            System.out.println("The circle area is " +
17                ((CircleFromSimpleGeometricObject)object).getArea());
18            System.out.println("The circle diameter is " +
19                ((CircleFromSimpleGeometricObject)object).getDiameter());
20        }
21        else if (object instanceof
22            RectangleFromSimpleGeometricObject) {
23            System.out.println("The rectangle area is " +
24                ((RectangleFromSimpleGeometricObject)object).getArea());
25        }
26    }
27 }

```

Problems @ Javadoc Declaration Console

<terminated> CastingDemo [Java Application] C:\Program Files\Ja

The circle area is 3.141592653589793

The circle diameter is 2.0

The rectangle area is 1.0



在父类构造函数中调用被重写的方法，可能会出现意想不到的结果。

调用哪个test
?

什么时候执行?

```
3 class Base1 {
4     public Base1() {
5         test();
6     }
7     public void test() {
8
9     }
10 }
11 public class Child extends Base1 {
12     private int a = 123;
13     public Child() {
14
15     }
16     public void test() {
17         System.out.println(a);
18     }
19
20     public static void main(String[] args) {
21         Child c = new Child();
22         c.test();
23     }
24 }
```

```
public Child(){
    ->super(); //此时a=0;
    -> a = 123;
    .....
}
```

```
Problems @ Javadoc Declaration Console
<terminated> Child [Java Application] C:\Program F
0
123
```



```

3 class Base1 {
4     private int a = 1;
5     public Base1() {
6         test();
7     }
8     public void test() {
9         System.out.println(a);
10    }
11 }
12 public class Child extends Base1 {
13     private int a = 123;
14     public Child() {
15
16     }
17     public void test() {
18         System.out.println(a);
19     }
20
21     public static void main(String[] args) {
22         Child c = new Child();
23         c.test();
24     }
25 }

```

Problems @ Javadoc Declaration Con

<terminated> Child [Java Application] C:\f

0

123

```

3 class Base1 {
4     private int a = 1;
5     public Base1() {
6         test();
7     }
8     public void
9         System.o
10    }

```

a (Base1)= 0

```

3 class Base1 {
4     private int a = 1;
5     public Base1() {
6         test();
7     }
8     public void
9         System.o
10    }
11 }
12 public class Child extends Base1 {
13     private int a = 123;
14     public Child() {
15
16     }
17     public void test() {
18         System.out.println(a);
19     }
20
21     public static void main(String[] args) {
22         Child c = new Child();
23         c.test();
24     }
25 }

```

a (Base1)= 1

a (Child)= 0

```

12 public class Child extends Base1 {
13     private int a = 123;
14     public Child() {
15
16     }
17     public void test() {
18         System.out.println(a);
19     }
20
21     public static void main
22         Child c = new Child
23         c.test();
24     }
25 }

```

a (Child)= 0

0

Java初始化构造函数调用顺序类初始化时构造函数调用顺序：

- （1）初始化对象的存储空间为零或null值；
- （2）调用父类构造函数；
- （3）按顺序分别调用类成员变量和实例成员变量的初始化表达式；
- （4）调用本身构造函数内容。



当子类重写父类非private的instance method时，则动态绑定执行方法。

当instance variable, static method, static variable重名时，如何访问？

重名时，表示有两个变量或方法，就看对象类型（subType则是子类, superType则是父类）。



```
public class Base {
    public static String s = "static_base";
    public String m = "base";
    public static void staticTest(){
        System.out.println("base static: "+s);
    }
}
```

```
public class Child extends Base {
    public static String s = "child_base";
    public String m = "child";
    public static void staticTest(){
        System.out.println("child static: "+s);
    }
}
```

```
public static void main(String[] args) {
    Child c = new Child();
    Base b = c;
    System.out.println(b.s);
    System.out.println(b.m);
    b.staticTest();
    System.out.println(c.s);
    System.out.println(c.m);
    c.staticTest();
}
```

static_base

base

base static: static_base

child_base

child

child static: child_base



```

3 class Base1 {
4     public int a = 1;
5     public Base1() {
6         test();
7     }
8     public void test() {
9         System.out.println(a);
10    }
11 }
12 public class Child extends Base1 {
13     public int a = 123;
14     public Child() {
15
16     }
17     public void test() {
18         System.out.println(a);
19     }
20
21     public static void main(String[] args) {
22         Child c = new Child();
23         c.test();
24         Base1 b = c;
25         System.out.println(b.a);
26         System.out.println(c.a);
27     }
28 }

```

Problems Javadoc Declaration Console

<terminated> Child [Java Application] C:\Program

```

0
123
1
123

```



```

3  class Base1 {
4      public int sum(int a, int b) {
5          System.out.println("base_int_int");
6          return a+b;
7      }
8  }
9  public class Child extends Base1 {
10     public long sum(long a, long b) {
11         System.out.println("child_long_long");
12         return a+b;
13     }
14
15     public static void main(String[] args) {
16         Child c = new Child();
17         int a=2;
18         int b=3;
19         c.sum(a,b);
20     }
21 }

```

```

- Problems - Variables - Declaration
<terminated> Child [Java Applicati
base_int_int

```



```

3 class Base1 {
4     public long sum(int a, long b) {
5         System.out.println("base_int_long");
6         return a+b;
7     }
8 }
9 public class Child extends Base1 {
10     public long sum(long a, long b) {
11         System.out.println("child_long_long");
12         return a+b;
13     }
14
15     public static void main(String[] args) {
16         Child c = new Child();
17         int a=2;
18         int b=3;
19         c.sum(a,b);
20     }
21 }

```

Problems	@ Javadoc	Declaration	Console
----------	-----------	-------------	---------

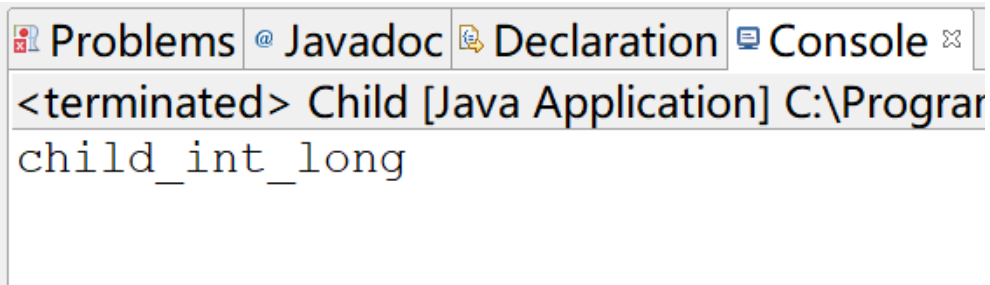
<terminated> Child [Java Application] C:\Program Files\J
base_int_long




```

3 class Base1 {
4     public long sum(int a, long b) {
5         System.out.println("base_int_long");
6         return a+b;
7     }
8 }
9 public class Child extends Base1 {
10     public long sum(int a, long b) {
11         System.out.println("child_int_long");
12         return a+b;
13     }
14
15     public static void main(String[] args) {
16         Child c = new Child();
17         int a=2;
18         int b=3;
19         c.sum(a,b);
20     }
21 }
22

```



The equals Method

The `equals()` method compares the contents of two objects. The **default implementation** of the `equals` method in the `Object` class is as follows:

```
public boolean equals(Object obj) {  
    return this == obj;  
}
```

For example, the `equals` method is overridden in the `Circle` class.

```
public boolean equals(Object o) {  
    if (o instanceof Circle) {  
        return radius == ((Circle)o).radius;  
    }  
    else  
        return false;  
}
```



NOTE

The `==` comparison operator is used for comparing two **primitive data type values** or for determining **whether two objects have the same references**.

The `equals` method is intended to test whether two objects **have the same contents**, provided that the method is modified in the defining class of the objects.

The `==` operator is stronger than the `equals` method, in that the `==` operator checks whether the two reference variables refer to the same object.



```
String str1 = new String("abc");
```

```
String str2 = new String("abc");
```

```
String str3 = "abc";
```

```
String str4 = "abc";
```

```
System.out.println(str1.equals(str2));
```

?

```
System.out.println(str1 == str2);
```

?

```
System.out.println(str3.equals(str4));
```

?

```
System.out.println(str3==str4);
```

?



The protected Modifier

The `protected` modifier can be applied on **data and methods** in a class.

A protected data or a protected method in a public class can be accessed by any class in the same package or its subclasses, even if the subclasses are in a different package.

`private`, `default`, `protected`, `public`

Visibility increases
→
`private`, `none` (if no modifier is used), `protected`, `public`



Accessibility Summary

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	-
default	✓	✓	-	-
private	✓	-	-	-



Visibility Modifiers

package p1;

```
public class C1 {  
    public int x;  
    protected int y;  
    int z;  
    private int u;  
  
    protected void m() {  
    }  
}
```

```
public class C2 {  
    C1 o = new C1();  
    can access o.x;  
    can access o.y;  
    can access o.z;  
    cannot access o.u;  
  
    can invoke o.m();  
}
```



```
public class C3  
    extends C1 {  
    can access x;  
    can access y;  
    can access z;  
    cannot access u;  
  
    can invoke m();  
}
```

package p2;

```
public class C4  
    extends C1 {  
    can access x;  
    can access y;  
    cannot access z;  
    cannot access u;  
  
    can invoke m();  
}
```

```
public class C5 {  
    C1 o = new C1();  
    can access o.x;  
    cannot access o.y;  
    cannot access o.z;  
    cannot access o.u;  
  
    cannot invoke o.m();  
}
```

Protected的一个用法：模板方法

```
3 class Base1 {
4     protected int currentStep;
5     protected void step1() {}
6     protected void step2() {}
7     public void action() {
8         this.currentStep=1;
9         step1();
10        this.currentStep=2;
11        step2();
12    }
13 }
14 public class Child extends Base1 {
15     protected void step1() {
16         System.out.println("child step " + this.currentStep);
17     }
18     protected void step2() {
19         System.out.println("child step " + this.currentStep);
20     }
21     public static void main(String[] args) {
22         Child c = new Child();
23         c.action();
24     }
25 }
26
```


A Subclass Cannot Weaken the Accessibility

A subclass may override a **protected** method in its superclass and change its visibility to **public**.

However, a subclass cannot weaken the accessibility of a method defined in the **superclass**. For example, if a method is defined as public in the superclass, it must be defined as public in the subclass.

```
3 class Base1 {  
4     protected void protect() {}  
5     public void open() {}  
6 }  
7 public class Child extends Base1 {  
8     //以下是不允许的，会有编译错误  
9     //private void protect() {}  
10    //protected void open() {}  
11    public void protect() {}  
12 }  
13
```



NOTE

The modifiers are used on classes and class members (data and methods), except that the final modifier can also be used on local variables in a method.

A final local variable is a constant inside a method.



The final Modifier

The final class **cannot be extended**:

```
final class Math {  
    ...  
}
```

The final variable is a constant:

```
final static double PI = 3.14159;
```

The final method **cannot be overridden** by its subclasses.



继承实现原理介绍

```
3 public class Base {
4     public static int s;
5     private int a;
6     static{
7         System.out.println("基类静态代码块, s: " + s);
8         s = 1;
9     }
10    {
11        System.out.println("基类实例代码块, a: " + a);
12        a = 1;
13    }
14    public Base() {
15        System.out.println("基类构造方法, a: " + a);
16        a = 2;
17    }
18    protected void step() {
19        System.out.println("base s: " + s + ", a: " + a);
20    }
21    public void action() {
22        System.out.println("start");
23        step();
24        System.out.println("end");
25    }
26 }
27
```



```
public class Child extends Base{
    public static int s;
    private int a;
    static{
        System.out.println("子类静态代码块, s: " + s);
        s = 10;
    }
    {
        System.out.println("子类实例代码块, a: " + a);
        a = 10;
    }
    public Child(){
        System.out.println("子类构造方法, a: " + a);
        a = 20;
    }
    protected void step(){
        System.out.println("child s: " + s + ", a: " + a);
    }
}
```



```

public static void main(String[] args) {
    System.out.println("---- new Child()");
    Child c = new Child();
    System.out.println("\n---- c.action()");
    c.action();
    Base b = c;
    System.out.println("\n---- b.action()");
    b.action();
    System.out.println("\n---- b.s: " + b.s);
    System.out.println("\n---- c.s: " + c.s);
}

```

类加载过程

第一次使用类时，才会加载类

- ☐ 分配内存保存类的信息
- ☐ 给类变量（静态变量）赋默认值
- ☐ 加载父类
- ☐ 设置父子关系
- ☐ 执行类初始化代码
 - ☐ 定义静态变量时的赋值语句
 - ☐ 静态初始化代码块

```

terminated Child (/java/

```

```

基类静态代码块, s: 0
子类静态代码块, s: 0
---- new Child()
基类实例代码块, a: 0
基类构造方法, a: 1
子类实例代码块, a: 0
子类构造方法, a: 10

```

类加载过程

对象创建过程

```

---- c.action()
start
child s: 10, a: 20
end

```

```

---- b.action()
start
child s: 10, a: 20
end

```

```

---- b.s: 1

```

```

---- c.s: 10

```

动态绑定

静态变量访问



若将Base和Child类中的private int a， 改为public int a， 且都增加

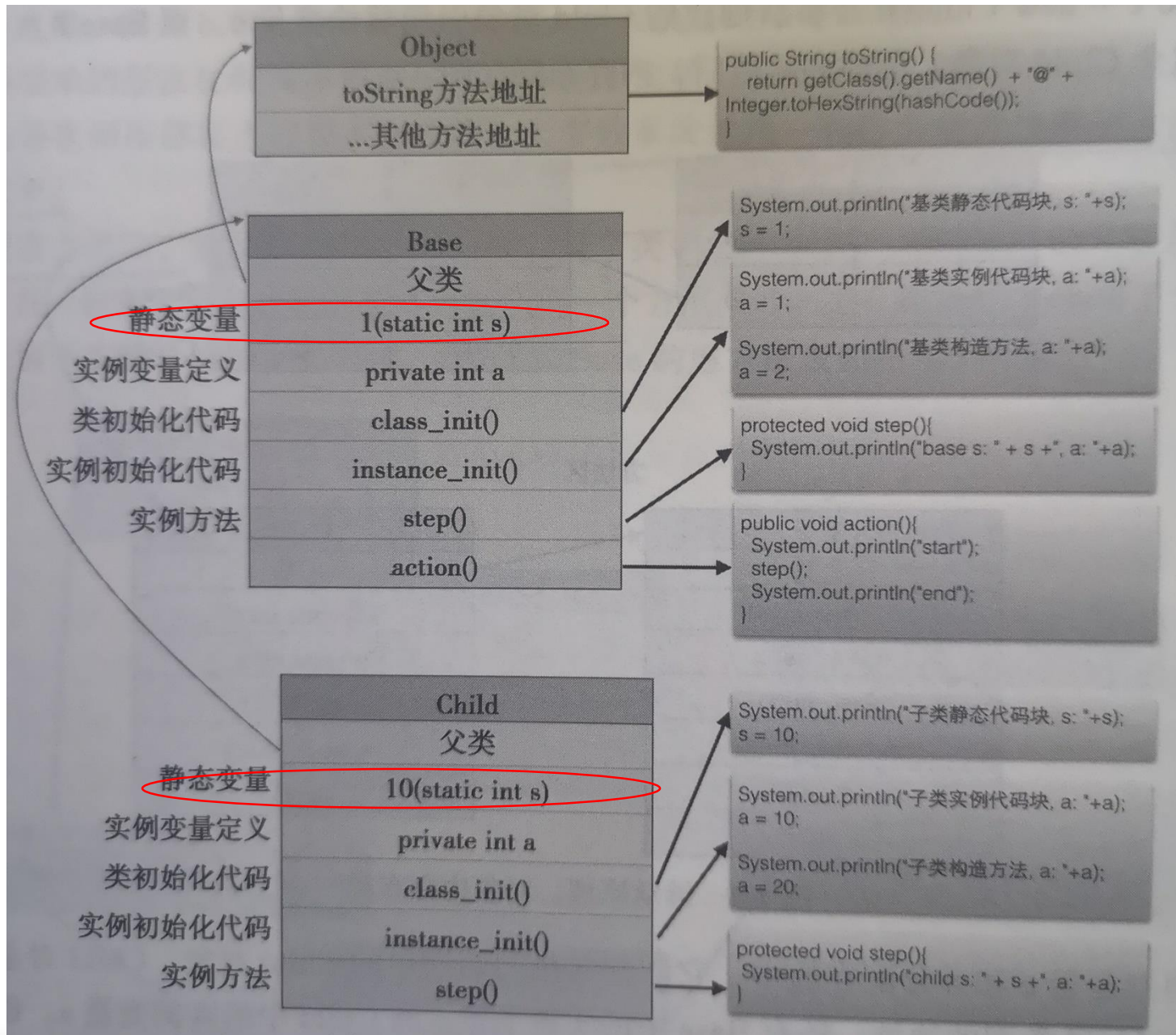
- public int getA(){return a;}

则：

- Child c = **new Child()**;
- Base b = c;
- System.out.println(c.getA());
- System.out.println(b.getA());
- System.out.println(c.a);
- System.out.println(b.a);



类信息内存布局

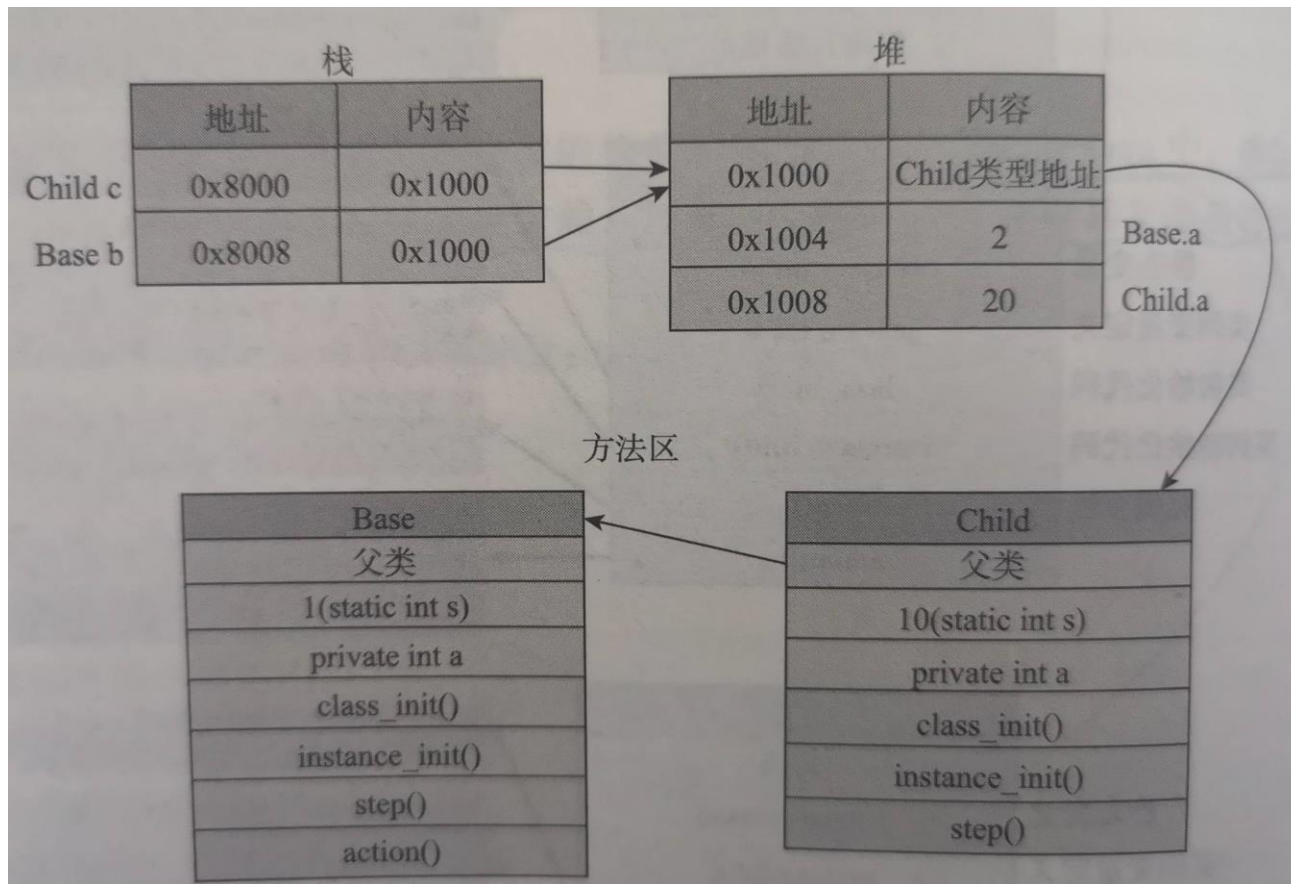


对象创建过程

- ❑ 分配内存
- ❑ 对所有实例变量赋默认值
- ❑ 执行实例初始化代码

Child c = new Child();

Base b = c;



方法调用过程 `c.action();`

- ❑ 查看c的类型，找到Child类型，在Child类型中找action方法，发现没有则到父类中去查找；
- ❑ 在父类Base中找到action方法，开始执行action方法；
- ❑ action先输出start，然后发现需要调用step方法，就从Child类型中寻找step方法；
- ❑ 在Child类型中找到step方法，则执行该方法，执行完后返回action方法；
- ❑ 继续执行action方法，输出end

方法调用过程 `b.action();`

- ❑ 由于b的类型实际上是Child，所以与c.action()一样的结果



继承是把双刃剑

继承是实现代码重用的有力手段，但它并非是完成这项工作的最佳手段。

原因在于：

- 继承打破了封装性，也就是子类依赖于其超类中特定功能的实现（由于动态绑定！）。而超类的实现有可能会随着发行版本的不同而有所变化。如果真的发生了变化，则子类有可能遭到破坏。



```
3 public class Base {
4     private static final int MAX_NUM=1000;
5     private int[] arr = new int[MAX_NUM];
6     private int count;
7     public void add(int number) {
8         if(count<MAX_NUM) {
9             arr[count++] = number;
10        }
11    }
12    public void addAll(int[] numbers) {
13        for(int num:numbers) {
14            add(num);
15        }
16    }
17 }
```



```

3 public class Child extends Base {
4     private long sum;
5     @Override
6     public void add(int number) {
7         super.add(number);
8         sum+=number;
9     }
10    @Override
11    public void addAll(int[] numbers) {
12        super.addAll(numbers);
13        for(int i=0;i<numbers.length;i++){
14            sum+=numbers[i];
15        }
16    }
17    public long getSum() {
18        return sum;
19    }
20
21    public static void main(String[] args) {
22        Child c = new Child();
23        c.addAll(new int[]{1,2,3});
24        System.out.println(c.getSum());
25    }
26 }

```

Why?

```

3 public class Base {
4     private static final int MAX_NUM=1000;
5     private int[] arr = new int[MAX_NUM];
6     private int count;
7     public void add(int number) {
8         if(count<MAX_NUM) {
9             arr[count++] = number;
10        }
11    }
12    public void addAll(int[] numbers) {
13        for(int num:numbers) {
14            add(num);
15        }
16    }
17 }

```

Problems @ Javado
 <terminated> Child
 12



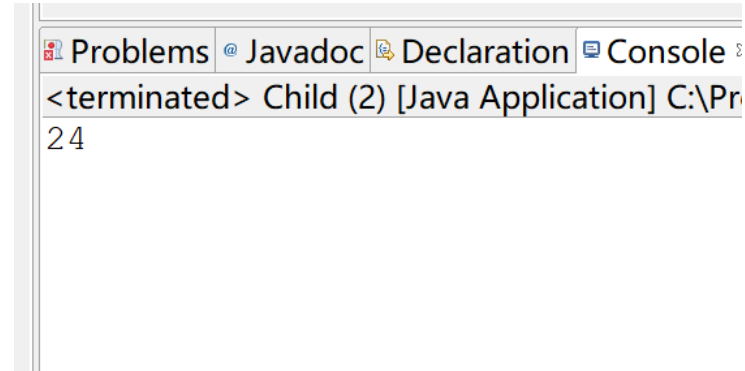
父类不能随意增加公开方法，因为父类增加就是给所有子类增加，而子类可能必须要重写该方法才能确保方法的正确性。

```
3 public class Base {
4     private static final int MAX_NUM=1000;
5     private int[] arr = new int[MAX_NUM];
6     private int count;
7     public void add(int number) {
8         if(count<MAX_NUM) {
9             arr[count++] = number;
10        }
11    }
12    public void addAll(int[] numbers) {
13        for(int num:numbers) {
14            add(num);
15        }
16    }
17    public void clear() {
18        for(int i=0;i<count;i++){
19            arr[i] = 0;
20        }
21        count = 0;
22    }
23 }
```

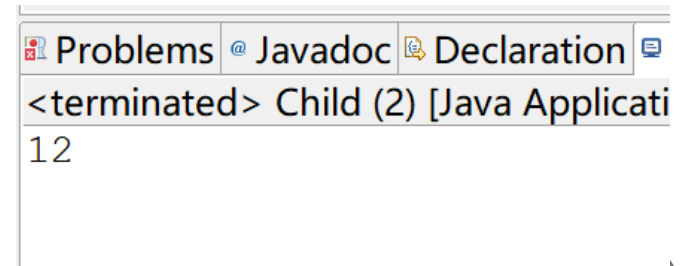


Child类自动拥有了clear方法，使用者可能会如下使用：

```
public static void main(String[] args) {  
    Child c = new Child();  
    c.addAll(new int[] {1, 2, 3});  
    c.clear();  
    c.addAll(new int[] {1, 2, 3});  
    System.out.println(c.getSum());  
}
```



```
public void clear() {  
    super.clear();  
    sum = 0;  
}
```



如何应对继承的双面性

避免使用继承

- 使用final
- 优先使用组合而非继承
- 使用接口

正确使用继承



组合（composition）：不继承现有的类，而是在新的类中增加一个私有域，并引用现有类的一个实例。

```
3 public class NewChild{
4     private Base base;
5     private long sum;
6
7     public NewChild(){
8         base = new Base();
9     }
10
11     public void add(int number){
12         base.add(number);
13         sum+=number;
14     }
15     public void addAll(int[] numbers){
16         base.addAll(numbers);
17         for(int i=0;i<numbers.length;i++){
18             sum+=numbers[i];
19         }
20     }
21     public long getSum(){
22         return sum;
23     }
24
25     public void clear(){
26         base.clear();
27         sum = 0;
28     }
29
30     public static void main(String[] args){
31         NewChild c = new NewChild();
32         c.addAll(new int[]{1,2,3});
33         c.clear();
34         c.addAll(new int[]{1,2,3});
35         System.out.println(c.getSum());
36     }
37 }
```

设计模式之装饰器模式

```
public interface FriedChicken {
```

```
    void makeFriedChicken();
```

```
}
```

```
public class KFC implements FriedChicken{
```

```
    @Override
```

```
    public void makeFriedChicken() {
```

```
        System.out.println("肯德基制作了一份炸鸡");
```

```
    }
```

```
}
```

```
public class McDonald implements FriedChicken{
```

```
    @Override
```

```
    public void makeFriedChicken() {
```

```
        System.out.println("麦当劳制作了一份炸鸡");
```

```
    }
```

```
}
```

```
public class FriedChickenDecorator implements FriedChicken{

    // 内部维护一个炸鸡类
    private FriedChicken friedChicken;

    // 通过构造方法把需要装饰的炸鸡类传进来
    public FriedChickenDecorator(FriedChicken friedChicken) {
        this.friedChicken = friedChicken;
    }

    // 增强方法
    @Override
    public void makeFriedChicken() {

        friedChicken.makeFriedChicken();
        System.out.println("检查炸鸡是否存在问题");
    }
}
```

```
public static void main(String[] args) {
```

```
    FriedChicken kfc = new KFC();  
    System.out.println("没增强前:");  
    kfc.makeFriedChicken();  
    System.out.println("=====");
```

```
    FriedChicken mcDonald = new McDonald();  
    System.out.println("没增强前:");  
    mcDonald.makeFriedChicken();  
    System.out.println("=====");
```

```
    // 把需要增强的类通过构造方法传进去
```

```
    FriedChicken kfcDecorator = new FriedChickenDecorator(kfc);  
    System.out.println("增强后:");  
    kfcDecorator.makeFriedChicken();  
    System.out.println("=====");
```

```
    // 把需要增强的类通过构造方法传进去
```

```
    FriedChicken mcDonaldDecorator = new FriedChickenDecorator(mcDonald);  
    System.out.println("增强后:");  
    mcDonaldDecorator.makeFriedChicken();
```

```
}
```



没增强前:

肯德基制作了一份炸鸡

=====

没增强前:

麦当劳制作了一份炸鸡

=====

原本KFC或者麦当劳的制作炸鸡方法

增强后:

肯德基制作了一份炸鸡

检查炸鸡是否存在问题

=====

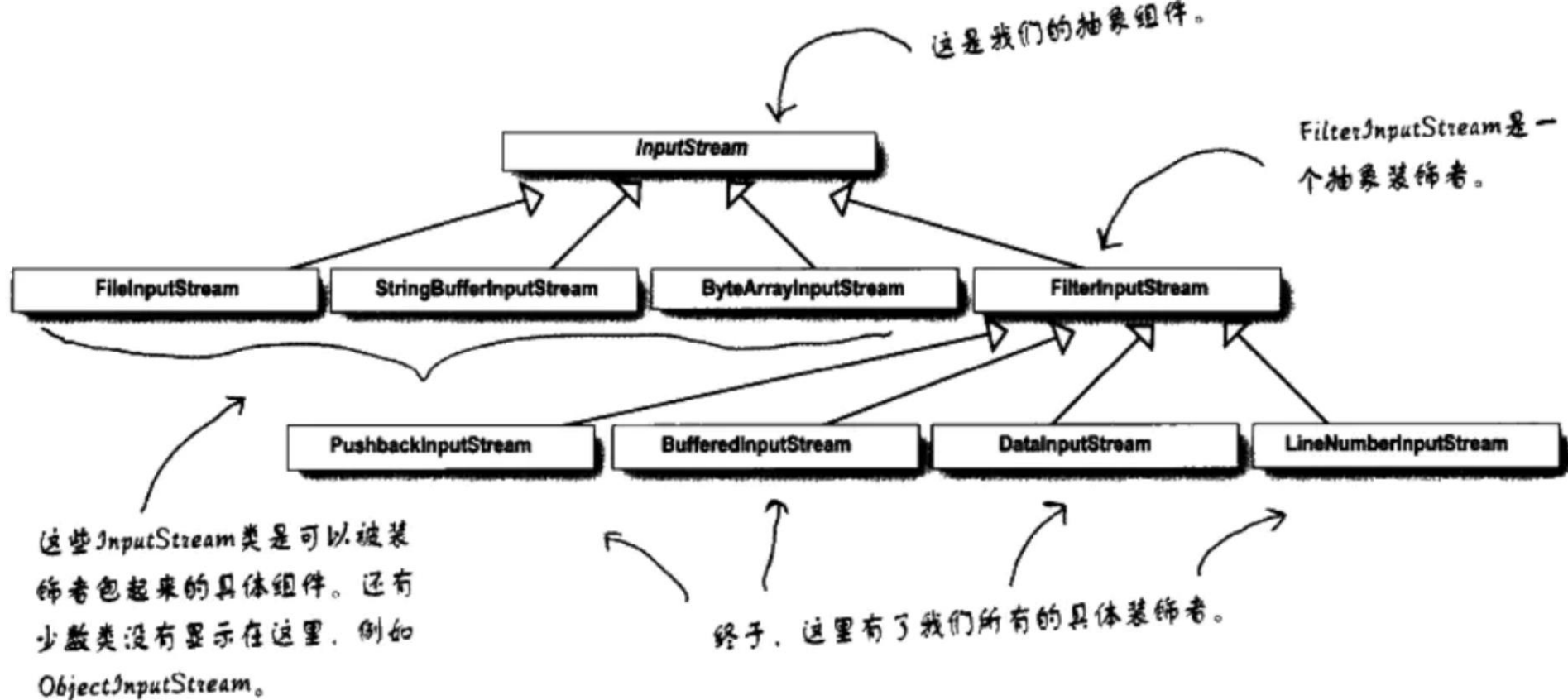
增强后:

麦当劳制作了一份炸鸡

检查炸鸡是否存在问题

走的装饰类的增强方法





```
public abstract class InputStream implements Closeable{
    public abstract int read() throws IOException;
}
```

```
public class FilterInputStream extends InputStream{
    protected volatile InputStream in;
    protected FilterInputStream(InputStream in) {
        this.in = in;
    }
    public int read() throws IOException {
        return in.read();
    }
}
```

Nested Class (嵌套类)

可以在一个类的内部定义另一个类，这种类称为嵌套类（nested classes）。

外面的类叫：外部类 (Outer Class);

内部的类，称为：内部类 (Inner Class)。

```
public class Outer
{
    private class Inner
    {
        // inner class instance variables
        // inner class methods

    } // end of inner class definition

    // outer class instance variables
    // outer class methods
}
```



Nested Class

Inner Class与包含它的Outer Class有比较密切的关系，而与其他类关系不大。

定义在类内部，可以实现对外部完全隐藏，可以有更好的封装性。

Inner Class只是Java编译器的概念，对于JVM，则不知道内部类这回事，因为每个内部类都会编译为一个独立的类，生成一个独立的class文件。

Inner Class可以方便访问外部类的私有变量，可以声明为private，从而实现对外完全隐藏。



Nested Class

4中内部类

- 静态内部类 (Static Inner Class)
- 成员内部类
- 方法内部类 (Local Class)
- 匿名内部类 (Anonymous Class)



Static Inner Class

Static Inner Class与Static Variable, Static Method一样, 带有static关键词。

```
3 public class Outer {
4     private static int shared = 100;
5     public static class StaticInner{
6         public void innerMethod() {
7             System.out.println("innerMethod, shared = " + shared);
8         }
9         public static void innerStaticMethod() {
10            System.out.println("innerStaticMethod, shared = " + shared);
11        }
12    }
13    public void test() {
14        StaticInner si = new StaticInner();
15        si.innerMethod();
16
17        StaticInner.innerStaticMethod();
18    }
19
20    public static void main(String[] args) {
21        Outer o = new Outer();
22        o.test();
23    }
24 }
```

Static Inner Class

Static Inner Class与一般的类差不多，可以有静态变量、静态方法、成员变量、成员方法、构造函数等。

Inner Class可以访问Outer Class的Static Variable和Method（无论private还是public），但不可访问Outer Class的成员变量和方法。

Outer Class可以直接使用Inner Class



```

public class Outer {
    private static int shared = 100;
    private int a = 1;
    public int b = 1;
    public static class StaticInner{
        public void innerMethod(){
            System.out.println("innerMethod, shared = " + shared);
            outerStaticMethod();

            System.out.println("Outer Private Variable a = " + a);
            System.out.println("Outer Private Variable b = " + b);
            outerMethod();
        }

        public static void innerStaticMethod(){
            System.out.println("innerStaticMethod, shared = " + shared);
        }
    }

    public void outerMethod(){
        System.out.println("outerMethod");
    }

    private static void outerStaticMethod(){
        System.out.println("outerStaticMethod");
    }

    public void test(){
        StaticInner si = new StaticInner();
        si.innerMethod();

        StaticInner.innerStaticMethod();
    }

    public static void main(String[] args) {
        Outer o = new Outer();
        o.test();
    }
}

```



Cannot make a static reference to the non-static field b
 1 quick fix available:
[Change 'b' to 'static'](#)
 Press 'F2' for focus

Cannot make a static reference to the non-static method outerMethod() from the type Outer
 1 quick fix available:
[Change 'outerMethod\(\)' to 'static'](#)
 Press 'F2' for focus



Static Inner Class

public的Static Inner Class可以被外部使用。

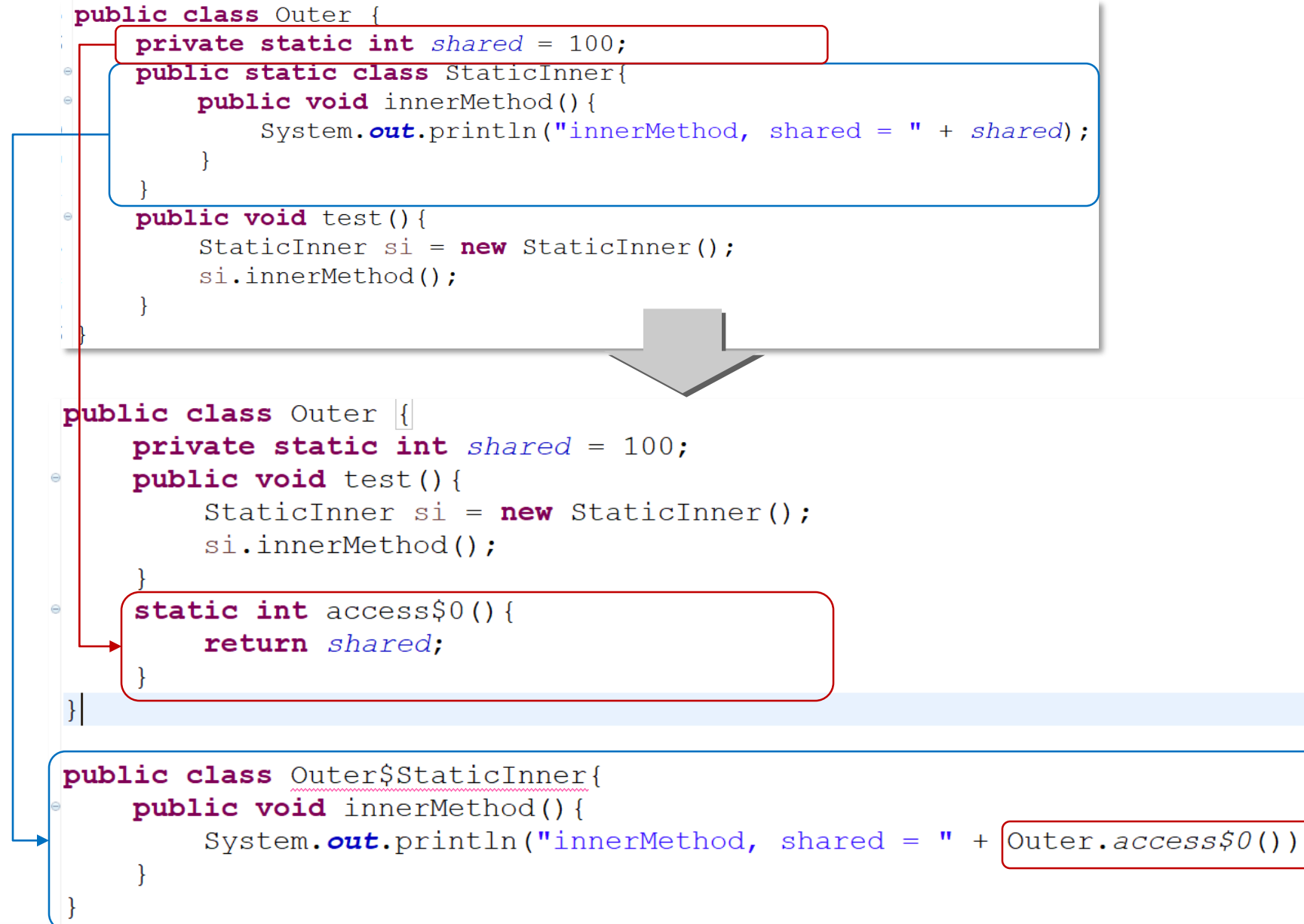
```
Outer.StaticInner si = new Outer.StaticInner();
```



Static Inner Class

内部实现机制

```
public class Outer {  
    private static int shared = 100;  
    public static class StaticInner {  
        public void innerMethod() {  
            System.out.println("innerMethod, shared = " + shared);  
        }  
    }  
    public void test() {  
        StaticInner si = new StaticInner();  
        si.innerMethod();  
    }  
}
```



```
public class Outer {  
    private static int shared = 100;  
    public void test() {  
        StaticInner si = new StaticInner();  
        si.innerMethod();  
    }  
    static int access$0() {  
        return shared;  
    }  
}
```

```
public class Outer$StaticInner {  
    public void innerMethod() {  
        System.out.println("innerMethod, shared = " + Outer.access$0());  
    }  
}
```

```

public class test.Outer {
    static {};
    Code:
        0: bipush          100
        2: putstatic      #10          // Field shared:I
        5: return

    public test.Outer();
    Code:
        0: aload_0
        1: invokespecial #15          // Method java/lang/Object."<init>":()V
        4: return

    public void test();
    Code:
        0: new              #20          // class test/Outer$StaticInner
        3: dup
        4: invokespecial #22          // Method test/Outer$StaticInner."<init>":()V
        7: astore_1
        8: aload_1
        9: invokevirtual #23          // Method test/Outer$StaticInner.innerMethod:()V
       12: return

    public static void main(java.lang.String[]);
    Code:
        0: new              #1           // class test/Outer
        3: dup
        4: invokespecial #30          // Method "<init>":()V
        7: astore_1
        8: aload_1
        9: invokevirtual #31          // Method test:()V
       12: new              #20          // class test/Outer$StaticInner
       15: dup
       16: invokespecial #22          // Method test/Outer$StaticInner."<init>":()V
       19: astore_2
       20: return

    static int access$0();
    Code:
        0: getstatic      #10          // Field shared:I
        3: ireturn
}

```

javap -c Outer.class



javap -c Outer\$StaticInner.class

```
public class test.Outer$StaticInner {
  public test.Outer$StaticInner();
  Code:
    0: aload_0
    1: invokespecial #8          // Method java/lang/Object."<init>":()V
    4: return

  public void innerMethod();
  Code:
    0: getstatic     #15         // Field java/lang/System.out:Ljava/io/PrintStream;
    3: new           #21         // class java/lang/StringBuilder
    6: dup
    7: ldc          #23         // String innerMethod, shared =
    9: invokespecial #25         // Method java/lang/StringBuilder."<init>":(Ljava/lang/String;)V
   12: invokestatic  #28         // Method test/Outer.access$0:()I
   15: invokevirtual #34         // Method java/lang/StringBuilder.append:(I)Ljava/lang/StringBuilder;
   18: invokevirtual #38         // Method java/lang/StringBuilder.toString:()Ljava/lang/String;
   21: invokevirtual #42         // Method java/io/PrintStream.println:(Ljava/lang/String;)V
   24: return
}
```



实际例子，Integer的静态嵌套类：

```
private static class IntegerCache {  
    static final int low = -128;  
    static final int high;  
    static final Integer cache[];  
  
    static {  
        // high value may be configured by property  
        int h = 127;  
        String integerCacheHighPropValue =  
            sun.misc.VM.getSavedProperty("java.lang.Integer.IntegerCache.high");  
        if (integerCacheHighPropValue != null) {  
            try {  
                int i = parseInt(integerCacheHighPropValue);  
                i = Math.max(i, 127);  
                // Maximum array size is Integer.MAX_VALUE  
                h = Math.min(i, Integer.MAX_VALUE - (-low) - 1);  
            } catch (NumberFormatException nfe) {  
                // If the property cannot be parsed into an int, ignore it.  
            }  
        }  
        high = h;  
  
        cache = new Integer[(high - low) + 1];  
        int j = low;  
        for(int k = 0; k < cache.length; k++)  
            cache[k] = new Integer(j++);  
  
        // range [-128, 127] must be interned (JLS7 5.1.7)  
        assert IntegerCache.high >= 127;  
    }  
  
    private IntegerCache() {}  
}
```

Integer类中

```
public static Integer valueOf(int i) {  
    if (i >= IntegerCache.low && i <= IntegerCache.high)  
        return IntegerCache.cache[i + (-IntegerCache.low)];  
    return new Integer(i);  
}
```



LinkedList静态内部类

```
private static class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;

    Node(Node<E> prev, E element, Node<E> next) {
        this.item = element;
        this.next = next;
        this.prev = prev;
    }
}

*/
transient Node<E> first;

/**
 * Pointer to last node.
 * Invariant: (first == null && last == null) ||
 *            (last.next == null && last.item != null)
 */
transient Node<E> last;

/**
 * Links e as last element.
 */
void linkLast(E e) {
    final Node<E> l = last;
    final Node<E> newNode = new Node<>(l, e, null);
    last = newNode;
    if (l == null)
        first = newNode;
    else
        l.next = newNode;
    size++;
    modCount++;
}
```

```
static class UnmodifiableCollection<E> implements Collection<E>, Serializable {  
    private static final long serialVersionUID = 1820017752578914078L;
```

```
    final Collection<? extends E> c;
```

```
    UnmodifiableCollection(Collection<? extends E> c) {  
        if (c==null)  
            throw new NullPointerException();  
        this.c = c;  
    }
```

```
    public int size() {return c.size();}  
    public boolean isEmpty() {return c.isEmpty();}  
    public boolean contains(Object o) {return c.contains(o);}  
    public Object[] toArray() {return c.toArray();}  
    public <T> T[] toArray(T[] a) {return c.toArray(a);}  
    public String toString() {return c.toString();}
```

```
    public Iterator<E> iterator() {  
        return new Iterator<E>() {  
            private final Iterator<? extends E> i = c.iterator();  
  
            public boolean hasNext() {return i.hasNext();}  
            public E next() {return i.next();}  
            public void remove() {  
                throw new UnsupportedOperationException();  
            }  
            @Override  
            public void forEachRemaining(Consumer<? super E> action) {  
                // Use backing collection version  
                i.forEachRemaining(action);  
            }  
        };  
    }
```

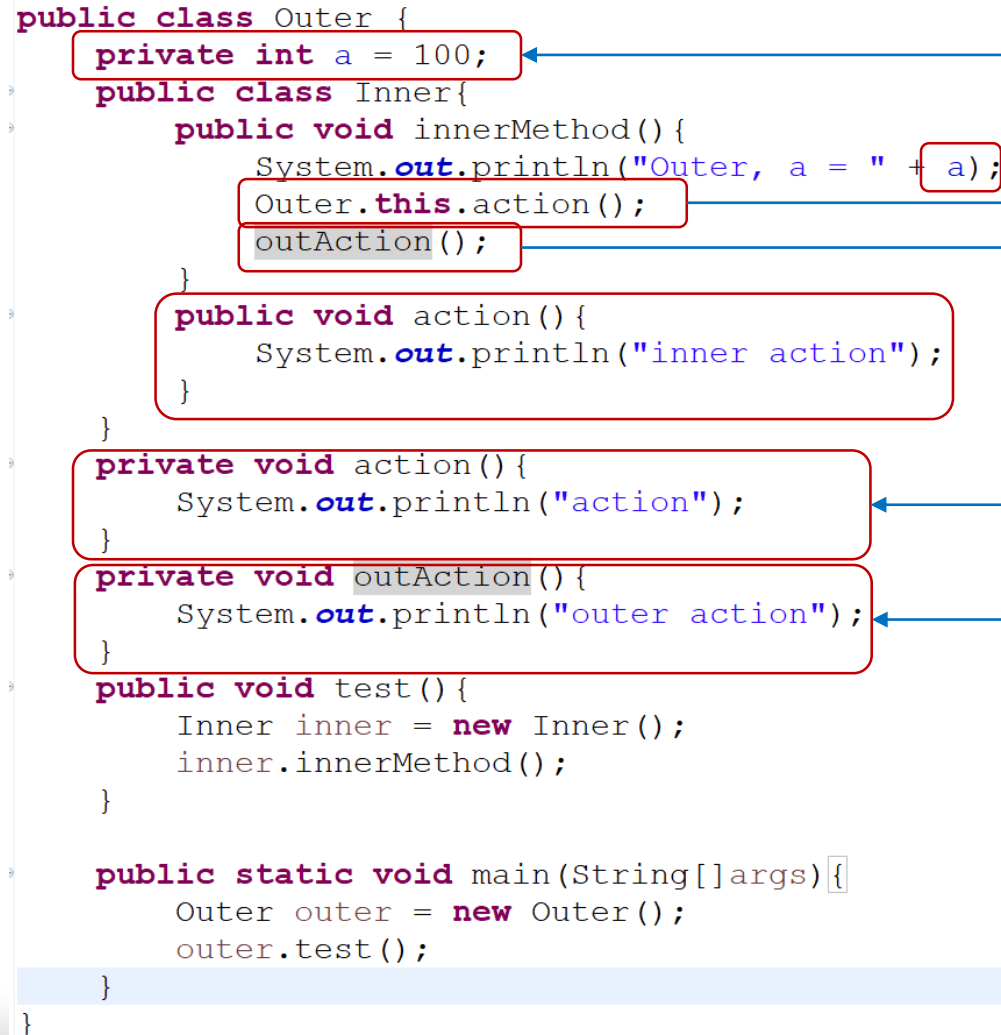
```
    public boolean add(E e) {  
        throw new UnsupportedOperationException();  
    },
```

Collections的静态嵌套类

成员内部类

与Static Inner Class相比，成员内部类没有static修饰符，含义大不同。

```
public class Outer {  
    private int a = 100;  
    public class Inner {  
        public void innerMethod() {  
            System.out.println("Outer, a = " + a);  
            Outer.this.action();  
            outAction();  
        }  
        public void action() {  
            System.out.println("inner action");  
        }  
    }  
    private void action() {  
        System.out.println("action");  
    }  
    private void outAction() {  
        System.out.println("outer action");  
    }  
    public void test() {  
        Inner inner = new Inner();  
        inner.innerMethod();  
    }  
  
    public static void main(String[] args) {  
        Outer outer = new Outer();  
        outer.test();  
    }  
}
```



成员内部类

在外部类内部，与静态内部类一样，可以直接使用内部类

```
public void test() {  
    Inner inner = new Inner();  
    inner.innerMethod();  
}
```

在外部使用，则与静态内部类不同

```
Outer.StaticInner si = new Outer.StaticInner();
```

VS.

```
public static void main(String[] args) {  
    Outer outer = new Outer();  
    Inner inner = outer.new Inner();  
}
```

成员内部类对象需要与一个外部类对象相连，所以不能
`Outer.Inner inner = new Outer.Inner();`



成员内部类

```
public class Outer {
    private int a = 100;
    public class Inner {
        public void innerMethod() {
            System.out.println("Outer, a = " + a);
            Outer.this.action();
            outAction();
        }
        public void action() {
            System.out.println("inner action");
        }
    }
    private void action() {
        System.out.println("action");
    }
    private void outAction() {
        System.out.println("outer action");
    }
    public void test() {
        Inner inner = new Inner();
        inner.innerMethod();
    }

    public static void main(String[] args) {
        Outer outer = new Outer();
        outer.test();
    }
}
```

```
public class Outer {
    private int a = 100;
    private void action() {
        System.out.println("action");
    }
    public void outAction() {
        System.out.println("outer action");
    }
    public void test() {
        Outer$Inner inner = new Outer$Inner(this);
        inner.innerMethod();
    }
}
```

```
static int access$0(Outer outer) {
    return outer.a;
}
static void access$1(Outer outer) {
    outer.action();
}
static void access$2(Outer outer) {
    outer.outAction();
}
```

```
public class Outer$Inner {
    final Outer outer;
    public Outer$Inner(Outer outer) {
        this.outer = outer;
    }
    public void innerMethod() {
        System.out.println("Outer, a = " + Outer.access$0(outer));
        Outer.access$1(outer);
        Outer.access$2(outer);
    }
    public void action() {
        System.out.println("inner action");
    }
}
```

相当于有个指向外部类对象的ref.

内部成员类

There are two main **advantages to inner classes**

- They can **make the outer class more self-contained** since they are defined inside a class
- **Both of their methods have access to each other's private methods and instance variables**

Using an inner class as a helping class is one of the most useful applications of inner classes

- If used as a helping class, an inner class should be marked private



LinkedHashMap类中有LinkedKeySet和LinkedValues内部类，可以帮助操作KeySet和Values

```
public Set<K> keySet() {  
    Set<K> ks;  
    return (ks = keySet) == null ? (keySet = new LinkedKeySet()) : ks;  
}  
  
final class LinkedKeySet extends AbstractSet<K> {  
    public final int size() { return size; }  
    public final void clear() { LinkedHashMap.this.clear(); }
```

```
public Collection<V> values() {  
    Collection<V> vs;  
    return (vs = values) == null ? (values = new LinkedValues()) : vs;  
}  
  
final class LinkedValues extends AbstractCollection<V> {  
    public final int size() { return size; }  
    public final void clear() { LinkedHashMap.this.clear(); }  
    public final Iterator<V> iterator() {  
        return new LinkedValueIterator();  
    }  
    public final boolean contains(Object o) { return containsValue(o); }
```

Inner and Outer Classes Have Access to Each Other's Private Members

Within the definition of a method of **an inner class**:

- It is legal to reference a private instance variable of the outer class
- It is legal to invoke a private method of the outer class
- Essentially, **the inner class has a hidden reference to the outer class**

Within the definition of a method of **the outer class**

- It is legal to reference a private instance variable of the inner class on an object of the inner class
- It is legal to invoke a (nonstatic) method of the inner class as long as an object of the inner class is used as a calling object

Within the definition of the inner or outer classes, the modifiers **public** and **private** are **equivalent**



```

public class Outer {
    int outer_x = 100;
    class Inner{
        public int y = 10;
        private int z = 9;
        int m = 5;
        public void display(){
            System.out.println("display outer_x:" + outer_x);
        }
        private void display2(){
            System.out.println("display outer_x:" + outer_x);
        }
    }
    void test(){
        Inner inner = new Inner();
        inner.display();
        inner.display2();
        //System.out.println("Inner y:" + y); //不能访问内部变量
        System.out.println("Inner y:" + inner.y); //可以访问
        System.out.println("Inner z:" + inner.z); //可以访问
        System.out.println("Inner m:" + inner.m); //可以访问
        InnerTwo innerTwo = new InnerTwo();
        innerTwo.show();
    }
    class InnerTwo{
        Inner innerx = new Inner();
        public void show(){
            //System.out.println(y); //不可访问Inner的y成员
            //System.out.println(Inner.y); //不可直接访问Inner的任何成员和方法
            innerx.display(); //可以访问
            innerx.display2(); //可以访问
            System.out.println(innerx.y); //可以访问
            System.out.println(innerx.z); //可以访问
            System.out.println(innerx.m); //可以访问
        }
    }
    public static void main(String args[]){
        Outer outer = new Outer();
        outer.test();
    }
}

```



Class with an Inner Class

Display 13.9 Class with an Inner Class (Part 1 of 2)

```
1 public class BankAccount
2 {
3     private class Money ← The modifier private in this line should
4     {                               not be changed to public.
5         private long dollars; ← However, the modifiers public and
6         private int cents;       private inside the inner class Money
                                   can be changed to anything else and it
                                   would have no effect on the class
                                   BankAccount.
7         public Money(String stringAmount)
8         {
9             abortOnNull(stringAmount);
10            int length = stringAmount.length();
11            dollars = Long.parseLong(
12                    stringAmount.substring(0, length - 3));
13            cents = Integer.parseInt(
14                    stringAmount.substring(length - 2, length));
15        }
16
17        public String getAmount()
18        {
19            if (cents > 9)
20                return (dollars + "." + cents);
21            else
22                return (dollars + ".0" + cents);
23        }
24    }
```

Class with an Inner Class

Display 13.9 Class with an Inner Class (*Part 1 of 2*) (continued)

```
23     public void addIn(Money secondAmount)
24     {
25         abortOnNull(secondAmount);
26         int newCents = (cents + secondAmount.cents)%100;
27         long carry = (cents + secondAmount.cents)/100;
28         cents = newCents;
29         dollars = dollars + secondAmount.dollars + carry;
30     }

31     private void abortOnNull(Object o)
32     {
33         if (o == null)
34         {
35             System.out.println("Unexpected null argument.");
36             System.exit(0);
37         }
38     }
39 }
```

The definition of the inner class ends here, but the definition of the outer class continues in Part 2 of this display.

Class with an Inner Class

Display 13.9 Class with an Inner Class (Part 2 of 2)

```
40     private Money balance;
41     public BankAccount()
42     {
43         balance = new Money("0.00");
44     }
45     public String getBalance()
46     {
47         return balance.getAmount();
48     }
49     public void makeDeposit(String depositAmount)
50     {
51         balance.addIn(new Money(depositAmount));
52     }
53     public void closeAccount()
54     {
55         balance.dollars = 0;
56         balance.cents = 0;
57     }
58 }
```

To invoke a nonstatic method of the inner class outside of the inner class, you need to create an object of the inner class.

This invocation of the inner class method `getAmount()` would be allowed even if the method `getAmount()` were marked as `private`.

Notice that the outer class has access to the private instance variables of the inner class.

This class would normally have more methods, but we have only included the methods we need to illustrate the points covered here.

Referring to a Method of the Outer Class

If a method is invoked in an inner class （当内部类调用一个函数时。。。）

- If the inner class has no such method, then it is assumed to be an invocation of the method of that name in the outer class
- If both the inner and outer class have a method with the same name, then it is assumed to be an invocation of the method in the inner class
- If both the inner and outer class have a method with the same name, and the intent is to invoke the method in the outer class, then the following invocation must be used:

OuterClassName.this.methodName()



Multiple Inner Classes

A class can have as many inner classes as it needs.

Inner classes have access **to each other's private members** as long as an object of the other inner class is used as the calling object.



The `.class` File for an Inner Class

Compiling any class in Java produces a `.class` file named `ClassName.class`

Compiling a class with one (or more) inner classes causes both (or more) classes to be compiled, and produces two (or more) `.class` files

- Such as `ClassName.class` and `ClassName$InnerClassName.class`



Nesting Inner Classes

It is legal to nest inner classes within inner classes

- The rules are the same as before, but the names get longer
- Given class **A**, which has public inner class **B**, which has public inner class **C**, then the following is valid:

```
A aObject = new A();
```

```
A.B bObject = aObject.new B();
```

```
A.B.C cObject = bObject.new C();
```



HashMap中的成员内部类

```
final class KeySet extends AbstractSet<K> {  
    public final int size() { return size; }  
    public final void clear() { HashMap.this.clear(); }  
    public final Iterator<K> iterator() { return new KeyIterator(); }  
    public final boolean contains(Object o) { return containsKey(o); }  
    public final boolean remove(Object key) {  
        return removeNode(hash(key), key, null, false, true) != null;  
    }  
    public final Spliterator<K> spliterator() {  
        return new KeySpliterator<>(HashMap.this, 0, -1, 0, 0);  
    }  
}
```

HashMap的域

HashMap的成员函数

```
transient Node<K,V>[] table;  
  
if (action == null)  
    throw new NullPointerException();  
if (size > 0 && (tab = table) != null) {  
    int mc = modCount;  
    for (int i = 0; i < tab.length; ++i) {  
        for (Node<K,V> e = tab[i]; e != null; e = e.next)  
            action.accept(e.key);  
    }  
    if (modCount != mc)  
        throw new ConcurrentModificationException();  
}
```

```
final class KeyIterator extends HashIterator  
    implements Iterator<K> {  
    public final K next() { return nextNode().key; }  
}
```

```

final class Values extends AbstractCollection<V> {
    public final int size()                { return size; }
    public final void clear()              { HashMap.this.clear(); }
    public final Iterator<V> iterator()    { return new ValueIterator(); }
    public final boolean contains(Object o) { return containsValue(o); }
    public final Splitter<V> splitter() {
        return new ValueSplitter<>(HashMap.this, 0, -1, 0, 0);
    }
    public final void forEach(Consumer<? super V> action) {
        Node<K,V>[] tab;
        if (action == null)
            throw new NullPointerException();
        if (size > 0 && (tab = table) != null) {
            int mc = modCount;
            for (int i = 0; i < tab.length; ++i) {
                for (Node<K,V> e = tab[i]; e != null; e = e.next)
                    action.accept(e.value);
            }
            if (modCount != mc)
                throw new ConcurrentModificationException();
        }
    }
}

```

```

public class ArrayList<E> extends AbstractList<E>
    implements List<E>, RandomAccess, Cloneable, java.io.Serializable
{
    ~/
    transient Object[] elementData; // non-private to simplify nested class access
    ~/
    private int size;

    private class Itr implements Iterator<E> {
        int cursor; // index of next element to return
        int lastRet = -1; // index of last element returned; -1 if no such
        int expectedModCount = modCount;

        public boolean hasNext() {
            return cursor != size;
        }

        @SuppressWarnings("unchecked")
        public E next() {
            checkForComodification();
            int i = cursor;
            if (i >= size)
                throw new NoSuchElementException();
            Object[] elementData = ArrayList.this.elementData;
            if (i >= elementData.length)
                throw new ConcurrentModificationException();
            cursor = i + 1;
            return (E) elementData[lastRet = i];
        }
    }
}

```

外部类对象的
引用

.....

方法内部类 (Local Class)

内部类可以定义在一个方法体中。

```
public class Outer {  
    private int a = 100;  
    public void test(final int param) {  
        final String str = "hello";  
        class Inner {  
            public void innerMethod() {  
                System.out.println("outer a = " + a);  
                System.out.println("param = " + param);  
                System.out.println("local var = " + str);  
            }  
        }  
        Inner inner = new Inner();  
        inner.innerMethod();  
    }  
    public static void main(String[] args) {  
        Outer o = new Outer();  
        o.test(1);  
    }  
}
```

```
<terminated> Outer (2) [Java Applicat
```

```
outer a = 100  
param = 1  
local var = hello
```



```

public class Outer {
    private int a = 100;
    public void test(int param) {
        String str = "hello";
        class Inner{
            public void innerMethod(){
                System.out.println("outer a = " + a);
                System.out.println("param = " + param);
                System.out.println("local var = " + str);
            }
        }
        Inner inner = new Inner();
        inner.innerMethod();
    }
    public static void main(String[] args) {
        Outer o = new Outer();
        o.test(1);
    }
}

```

Cannot refer to the non-final local variable str defined in an enclosing scope

1 quick fix available:

- Change modifier of 'str' to final

Press 'F2' for focus

Local Class

Local Class定义在方法中，只能在定义的方法内用。

如果方法是instance method, 除了静态变量和方法，内部类还可以直接访问外部类的instance variable和method。

如果方法是static method，则内部类只能访问外部类的static variables和static method。

Local Class可以访问方法的参数和局部变量（但必须是final的）



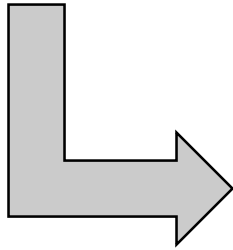
方法的局部变量位于栈上，只存在于该方法的生命期内。当一个方法结束，其栈结构被删除，局部变量成为历史。**但是该方法结束之后，在方法内创建的内部类对象可能仍然存在于堆中！**例如，如果对它的引用被传递到其他某些代码，并存储在一个成员变量内。正因为不能保证局部变量的存活期和方法内部类对象的一样长，所以内部类对象不能使用它们（非final的变量）。

```
class Outer {  
    public void doSomething(){  
        final int a =10;  
        class Inner{  
            public void seeOuter(){  
                System.out.println(a);  
            }  
        }  
        Inner in = new Inner();  
        in.seeOuter();  
    }  
    public static void main(String[] args) {  
        Outer out = new Outer();  
        out.doSomething();  
    }  
}
```



Local Class

```
public class Outer {
    private int a = 100;
    public void test(final int param) {
        final String str = "hello";
        class Inner {
            public void innerMethod() {
                System.out.println("outer a = " + a);
                System.out.println("param = " + param);
                System.out.println("local var = " + str);
            }
        }
        Inner inner = new Inner();
        inner.innerMethod();
    }
}
```



Local Class访问方法中的参数和局部变量时，这些变量需要声明为final。其实方法内部类中使用的是自己的实例变量

```
public class Outer{
    private int a = 100;
    public void test(final int param) {
        final String str = "hello";
        OuterInner inner = new OuterInner(this,param);
        inner.innerMethod();
    }

    static int access$0(Outer outer) {
        return outer.a;
    }
}

public class OuterInner{
    Outer outer;
    int param;
    OuterInner(Outer outer, int param) {
        this.outer = outer;
        this.param = param;
    }
    public void innerMethod() {
        System.out.println("outer a = " + Outer.access$0(outer));
        System.out.println("param = " + param);
        System.out.println("local var = " + "hello");
    }
}
```

Local Class

如果在Local Class中，想要修改外部变量，怎么办？

```
public class Outer{
    public void test() {
        final String[] str = new String[]{"hello java"};
        class Inner{
            public void innerMethod(){
                str[0] = "hello world";
            }
        }
        Inner inner = new Inner();
        inner.innerMethod();
        System.out.println(str[0]);
    }

    public static void main(String[] args) {
        Outer o = new Outer();
        o.test();
    }
}
```

匿名内部类 (Anonymous Class)

匿名内部类没有单独的类定义，它在创建对象的同时定义类。

```
new 父类(参数列表){  
    //匿名内部类实现部分  
}
```



构造函数的参数

```
new 父接口(){  
    //匿名内部类实现部分  
}
```



Anonymous Class

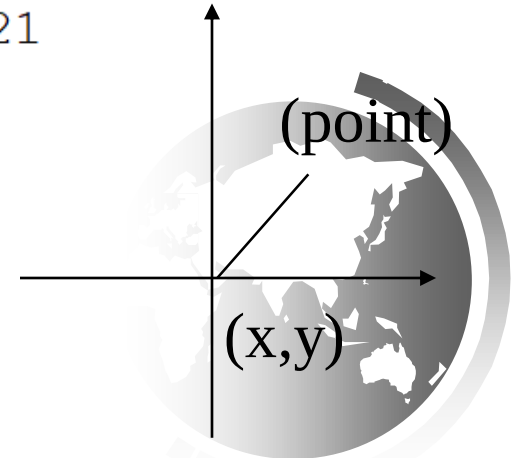
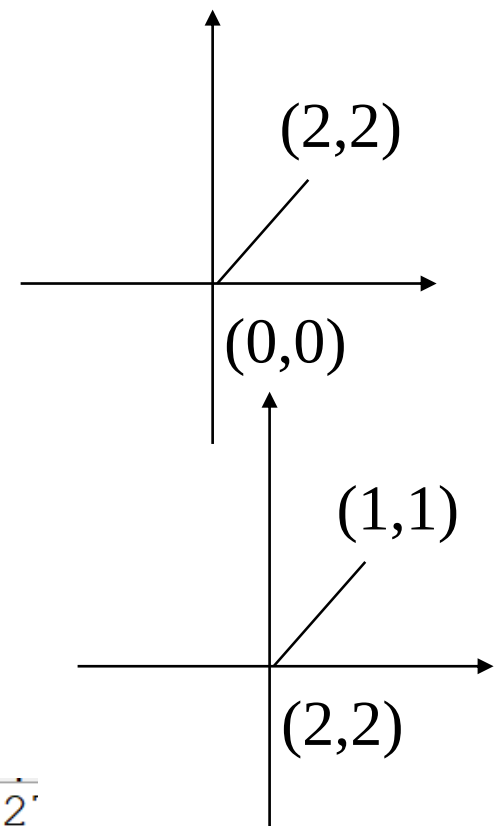
```

public class Outer{
    public void test(){
        Point p1 = new Point(2,2);
        System.out.println("p1.distance() = " + p1.distance());
    }
    public void test(final int x, final int y){
        Point p2 = new Point(2,2){
            @Override
            public double distance(){
                return distance(new Point(x,y));
            }
        };
        System.out.println("p2.distance() = " + p2.distance());
    }
    public static void main(String[] args){
        Outer o = new Outer();
        o.test();
        o.test(1,1);
    }
    class Point{
        private int x,y;
        public Point(int x, int y){this.x = x; this.y = y;}
        public int getX(){return x;}
        public int getY(){return y;}
        public double distance(Point point){
            return Math.sqrt(Math.pow(this.x-point.getX(),2)
                +Math.pow(this.y-point.getY(), 2));
        }
        public double distance(){
            return distance(new Point(0,0));
        }
    }
}

```

p1.distance() = 2.828427

p2.distance() = 1.414214



Anonymous Class

匿名内部类只能被使用一次，用来创建一个对象。它没有名字，也没有构造函数，但可以根据参数列表，调用对应的父类构造方法。

可以定义instance variable和method

可以有初始化代码块

与方法内部类一样，可以访问外部类的所有变量和方法，可以访问方法中的final参数和局部变量。



内部实现机制

```
class Point{
    private int x,y;
    public Point(int x, int y){this.x = x; this.y = y;}
    public int getX(){return x;}
    public int getY(){return y;}
    public double distance(Point point){
        return Math.sqrt(Math.pow(this.x-point.getX(),2)
            +Math.pow(this.y-point.getY(), 2));
    }
    public double distance(){
        return distance(new Point(0,0));
    }
}
```

```
public class Outer$1 extends Point{
    int x2;
    int y2;
    Outer outer;
    Outer$1(Outer outer, int x1,int y1, int x2, int y2){
        super(x1,y1);
        this.outer=outer;
        this.x2 = x2;
        this.y2 = y2;
    }
    @Override
    public double distance(){
        return distance(new Point(this.x2,this.y2));
    }
}
```

要求final

```
public class Outer{
    public void test(){
        Point p1 = new Point(2,2);
        System.out.println("p1.distance() = " + p1.distance());
    }
    public void test(final int x, final int y){
        Point p2 = new Outer$1(this,2,2,x,y);
        System.out.println("p2.distance() = " + p2.distance());
    }
    public static void main(String[]args){
        Outer o = new Outer();
        o.test();
        o.test(1,1);
    }
}
```


Arrays类

```
public static <T> void sort(T[] a, Comparator<? super T> c) {  
    if (c == null) {  
        sort(a);  
    } else {  
        if (LegacyMergeSort.userRequested)  
            legacyMergeSort(a, c);  
        else  
            TimSort.sort(a, 0, a.length, c, null);  
    }  
}
```

```
public void sortIgnoreCase(String [] strs) {  
    Arrays.sort(strs, new Comparator<String>() {  
        @Override  
        public int compare(String o1, String o2) {  
            return o1.compareToIgnoreCase(o2);  
        }  
    });  
}
```

```
--  
private static void mergeSort(Object[] src,  
                               Object[] dest,  
                               int low, int high, int off,  
                               Comparator c) {  
    int length = high - low;  
  
    // Insertion sort on smallest arrays  
    if (length < INSERTIONSORT_THRESHOLD) {  
        for (int i=low; i<high; i++)  
            for (int j=i; j>low && c.compare(dest[j-1], dest[j]) > 0;  
                swap(dest, j, j-1);  
        return;  
    }  
  
    // Recursively sort halves of dest into src  
    int destLow = low;  
    int destHigh = high;  
    low += off;  
    high += off;  
    int mid = (low + high) >>> 1;  
    mergeSort(dest, src, low, mid, -off, c);  
    mergeSort(dest, src, mid, high, -off, c);  
  
    // If list is already sorted, just copy from src to dest.  
    // optimization that results in faster sorts for nearly ordered lists  
    if (c.compare(src[mid-1], src[mid]) <= 0) {  
        System.arraycopy(src, low, dest, destLow, length);  
        return;  
    }  
  
    // Merge sorted halves (now in src) into dest  
    for (int i = destLow, p = low, q = mid; i < destHigh; i++) {
```



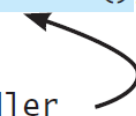
Inner Class Listeners

A listener class is designed specifically to create a listener object for a GUI component (e.g., a button). It will not be shared by other applications. So, **it is appropriate to define the listener class inside the frame class as an inner class.**



Anonymous Inner Classes

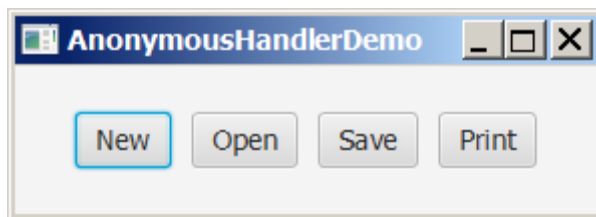
```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new EnlargeHandler());  
}  
  
class EnlargeHandler  
    implements EventHandler<ActionEvent> {  
    public void handle(ActionEvent e) {  
        circlePane.enlarge();  
    }  
}
```



(a) Inner class EnlargeListener

```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new class EnlargeHandler  
            implements EventHandler<ActionEvent>() {  
                public void handle(ActionEvent e) {  
                    circlePane.enlarge();  
                }  
            }  
    );  
}
```

(b) Anonymous inner class



AnonymousHandlerDemo

Run

上述例子都是针对接口编程的例子，也是都是一种回调的例子。

将程序分为保持不变的主体，和针对具体情况的可变逻辑，通过回调的方式进行协助，是计算机程序的一种常用实践。匿名内部类是实现回调接口的一种简便方式。

